XTART

FORMACIÓN PROFESIONAL

PROYECTO

EL DESPERTAR DEL HÉROE

AVENTURA TEXTUAL EN JAVA

1.ª EVALUACIÓN · 1.º DAM

DESARROLLO DE APLICACIONES MULTIPLATAFORMA

Título de la tarea: El despertar del héroe · Aventura textual en Java

Curso: 1.º de Desarrollo de Aplicaciones Multiplataforma

Módulos: Programación, entornos de desarrollo y lenguaje de marcas.

Modalidad: Trabajo individual · 1.ª evaluación

Entrega y defensa: según el calendario comunicado por el profesor

1. OBJETIVO GENERAL

Diseñar y desarrollar un juego de aventura por turnos que se ejecute en consola y permita crear un héroe, combatir, utilizar pociones y consultar las puntuaciones obtenidas durante la ejecución. El programa deberá ser comprensible, estar organizado en métodos y responder de forma controlada a las entradas del jugador.

Cada estudiante asumirá el papel de desarrollador de un prototipo para un estudio de videojuegos. El encargo comprende el análisis de las reglas, el diseño del algoritmo, la programación en Java, las pruebas y la entrega de una memoria técnica. El resultado se demostrará y defenderá individualmente.

2. OBJETIVOS ESPECÍFICOS

Aplicar variables, constantes, tipos de datos, operadores y entrada y salida por consola para representar el estado de una partida. Utilizar estructuras condicionales y repetitivas para controlar el menú, los turnos y el final de la aventura.

Dividir el problema en métodos estáticos con parámetros y valores de retorno. Utilizar arrays de tamaño fijo, validar los datos introducidos, comprobar casos límite y registrar la evolución del desarrollo mediante Git. La memoria y la defensa deberán justificar las decisiones y demostrar la comprensión del código.

3. APRENDIZAJES Y EVIDENCIAS

El proyecto integra los fundamentos de programación correspondientes a una primera evaluación. Los siguientes aprendizajes se comprobarán a través del programa, las pruebas, la memoria y la defensa.

Representación de la información

Seleccionar tipos de datos y constantes para nombre, vida, ataque, pociones y puntuación. Explicar el significado de cada variable y evitar valores mágicos.

Control del flujo

Construir menús con bucles de repetición y switch, resolver decisiones con if-else y controlar el combate mediante while. Recorrer arrays.

Descomposición del problema

Organizar la aplicación en métodos estáticos que reciban datos y devuelvan resultados. Explicar el alcance de las variables y el paso de parámetros.

Tratamiento de datos y errores

Gestionar un inventario y un ranking mediante arrays, limitar sus posiciones válidas y validar las entradas antes de utilizarlas.

Desarrollo y comunicación técnica

Mantener un historial progresivo en Git, documentar las pruebas y explicar el funcionamiento con una demostración reproducible.

Alcance de la primera evaluación

Se utilizará una clase principal de Java con métodos estáticos y datos básicos. No se exigen clases de dominio, herencia, polimorfismo, colecciones dinámicas, excepciones. El inventario y el ranking permanecerán en ficheros XML. Las ampliaciones voluntarias no sustituyen los requisitos comunes ni otorgan puntos adicionales.

4. DESCRIPCIÓN DEL PROYECTO

4.1. El encargo del estudio

El estudio desea validar un prototipo de El despertar del héroe antes de ampliar su desarrollo. El jugador deberá superar tres encuentros consecutivos con enemigos de dificultad creciente. La aventura finalizará al vencer al tercer enemigo, perder toda la vida o abandonar la partida. El estudiante decidirá la ambientación, los nombres y los mensajes, manteniendo las reglas comunes.

4.2. Inicio y navegación

El menú principal incluirá *Nueva partida, Consultar ranking, Ayuda y Salir*. Se implementará con do-while y switch y volverá a mostrarse al terminar una partida. La ayuda explicará reglas, acciones y puntuación. El nombre del héroe no podrá quedar vacío. Cada nueva partida reiniciará todos los datos del héroe.

4.3. Datos y reglas comunes

El héroe comenzará con 100 puntos de vida y tres pociones de 25 puntos de curación. Su ataque causará entre 10 y 20 puntos de daño, ambos incluidos. Los enemigos tendrán 40, 60 y 80 puntos de vida y causarán entre 5 y 15 puntos de daño. La vida nunca será negativa ni superará 100 tras curarse. Estos valores se definirán mediante constantes.

4.4. Combate por turnos

Un método simularCombate, o equivalente, gestionará cada encuentro mediante while. En cada turno se podrá atacar, usar una poción o abandonar. El daño se calculará con Math.random(). Si el enemigo sobrevive a una acción válida de ataque o curación, responderá una vez. Una opción inválida o intentar curarse sin pociones o con la vida completa no consumirá turno ni provocará un ataque enemigo. Un enemigo derrotado no podrá atacar.

4.5. Inventario con arrays

Se representarán las tres pociones mediante un array de tamaño fijo y un contador de elementos disponibles. El programa permitirá consultar el inventario durante el combate, recorrerá sus posiciones ocupadas y retirará una poción al consumirla. La consulta no consumirá turno. Se impedirá consumir posiciones vacías o acceder fuera de los límites.

4.6. Puntuación y ranking en memoria

Cada enemigo derrotado otorgará 100 puntos. Al ganar la aventura se añadirá la vida restante como bonificación. La puntuación será 100 × enemigos derrotados y, solo en caso de victoria completa, se sumará la vida final. Toda partida iniciada y terminada, incluso por abandono, generará un registro con nombre y puntos.

4.7. Organización y consulta del ranking

Se utilizarán dos arrays paralelos, uno de nombres y otro de puntuaciones, con capacidad de diez registros. Se mantendrán ordenados de mayor a menor puntuación sin desasociar los nombres. Cuando estén completos, solo entrará una puntuación superior a la décima. En los empates se conservará antes el registro más antiguo. Un ranking vacío mostrará un mensaje informativo. El ranking se conservará entre partidas en un fichero XML.

4.8. Modularidad y legibilidad

Se separarán, al menos, las responsabilidades de mostrar el menú, leer opciones válidas, iniciar una partida, resolver un combate, consultar o usar el inventario y registrar o mostrar el ranking. Los métodos recibirán los datos necesarios; se evitará concentrar el programa en main y utilizar variables globales mutables como solución general. Se valorará comprender que los parámetros primitivos y las referencias a arrays tienen comportamientos diferentes.

4.9. Entradas y pruebas

Las opciones no numéricas, fuera de rango y los nombres vacíos deberán tratarse sin cerrar el programa inesperadamente. Puede utilizarse Scanner.hasNextInt() o una estrategia equivalente. Se documentarán al menos ocho casos: nombre vacío, entrada no numérica, opción fuera de rango, victoria, derrota, uso de pociones, abandono y ranking. Se añadirán comprobaciones de empate, ranking lleno y reinicio de partida. En el combate aleatorio se comprobarán rangos y reglas, sin exigir un daño exacto.

4.10. Herramientas y control de versiones

El desarrollo se realizará con Java y el entorno utilizado en clase. Git registrará avances reales desde el inicio, con mensajes descriptivos y commits relacionados con tareas concretas. Se incluirá un .gitignore para archivos .class, carpetas bin o target y configuración local del IDE. La entrega conservará el historial mediante la carpeta .git o un repositorio remoto accesible al profesor.

5. CONTROL Y SEGUIMIENTO

El profesor revisará durante las clases el análisis, el menú y el combate, la integración del inventario y el ranking y las pruebas finales. Cada estudiante conservará sus avances y justificará los cambios. La fecha de entrega y los turnos de defensa se comunicarán en el aula virtual.

6. ESTRUCTURA DEL TRABAJO

La entrega incluirá el código fuente, una memoria técnica y una presentación de apoyo. Las capturas acompañarán a las explicaciones y no las sustituirán.

Apartado de la memoria	Contenido que debe desarrollar
1. Portada e índice	Título, estudiante, curso, módulo, fecha y páginas.
2. Introducción y análisis	Encargo, reglas, alcance y necesidades del jugador.
3. Diseño de la solución	Pseudocódigo o diagrama, variables, constantes, arrays y métodos.
4. Implementación	Menú, combate, inventario, puntuación y ranking; decisiones justificadas.
5. Pruebas	Entrada, resultado esperado y obtenido, evidencias, errores y correcciones.
6. Desarrollo con Git	Evolución del trabajo y explicación de commits representativos.
7. Uso de IA	Herramientas, instrucciones, contenido incorporado y comprobaciones; o declaración de no uso.
8. Conclusiones y recursos	Resultados, dificultades, aprendizajes, mejoras y fuentes consultadas.

7. FORMATO Y ENTREGA

Se entregará un .Zip con el enlace al repositorio de git donde debe estar todo el código del proyecto planteado. Contendrá el proyecto Java con sus fuentes, la memoria en PDF, la presentación en PDF o PPTX y una guía de ejecución con la versión de Java y los pasos para compilar y ejecutar.
En caso de no ser accessible dicho repositorio en el momento de la revisión, el proyecto se dará por no superado siendo necesaria su recuperación en la siguiente convocatoria.

La memoria tendrá portada, índice y numeración, con Arial o Times New Roman de 12 puntos e interlineado de 1,5. Las tablas podrán ajustar su formato para mantener la legibilidad. El código seguirá camelCase, indentación consistente y comentarios útiles; los métodos clave se documentarán con finalidad, parámetros y retorno cuando corresponda.

8. NOMENCLATURA DE ENTREGA

El paquete se denominará DAM_DESPERTAR_HEROE_Apellidos_Nombre.zip. La memoria y la presentación utilizarán el mismo nombre base con los sufijos _Memoria y _Presentacion. Las carpetas y los archivos técnicos tendrán nombres descriptivos.

9. PRESENTACIÓN ORAL

La defensa será individual y se apoyará en una presentación breve y una demostración en consola. El estudiante explicará las reglas, el diseño de los datos, los métodos y las decisiones adoptadas. Mostrará el menú, un combate, el consumo de pociones y la actualización del ranking entre partidas.

El profesor podrá preguntar sobre cualquier parte de la entrega y solicitar una modificación breve, como cambiar un rango de daño, explicar un recorrido de array o ajustar una validación. Se valorarán la comprensión, el razonamiento y la capacidad para localizar y modificar el código propio.

10. NORMAS Y USO DE INTELIGENCIA ARTIFICIAL

El trabajo es individual. Se permite utilizar documentación y herramientas de IA como apoyo, siempre que el estudiante comprenda, compruebe y pueda explicar todo lo entregado. La memoria identificará la herramienta, la tarea, ejemplos de instrucciones, los resultados incorporados o modificados y las verificaciones realizadas. Si no se ha usado IA, se indicará expresamente.

El uso de inteligencia artificial sin comprender el código, las explicaciones o cualquier otro contenido que proporcione y se incorpore al trabajo supondrá el suspenso directo del proyecto, con independencia de la puntuación obtenida en la rúbrica. La comprensión se comprobará durante el seguimiento y la defensa individual mediante preguntas, explicaciones y modificaciones breves del trabajo entregado.

11. RÚBRICA DE EVALUACIÓN

La rúbrica adjunta forma parte del proyecto. Contiene diez criterios con el mismo peso y cinco niveles: 1, 0,75, 0,5, 0,25 y 0 puntos por criterio. La suma proporciona la calificación sobre 10. Se asignará el nivel más alto cuyo descriptor se cumpla por completo, a partir del código, las pruebas, la memoria, el historial y la defensa.

Los criterios son: datos y operadores; flujo y menús; combate; inventario con arrays; ranking en memoria; modularidad; validación y pruebas; Git; documentación y estilo; y demostración y comprensión. Se evaluarán únicamente los requisitos comunicados. Las mejoras no compensan carencias en los requisitos comunes.

La condición sobre el uso de IA sin comprensión se aplica con independencia de la suma de la rúbrica. No implica que se penalice el mero uso declarado de estas herramientas.